

Leakage-safe IoT intrusion detection with White Shark Optimizer-based feature selection

Abdulla Amin Aburomman^a , Marwan Alakhras^a, Mamun Bin Ibne Reaz^{b,c}

^a Department of Cybersecurity and Cloud Computing, Faculty of Information Technology, Applied Science Private University (ASU), Amman, Jordan

^b Faculty of Electronic Engineering and Technology, Universiti Malaysia Perlis (UniMAP), 02600 Arau, Perlis, Malaysia

^c Institute of Microengineering and Nanoelectronics, Universiti Kebangsaan Malaysia, 43600 Bangi, Selangor, Malaysia

ARTICLE INFO

Dataset link: <https://www.unb.ca/cic/datasets/iotdiad-2024.html>

Keywords:

IoT security
Intrusion detection
Feature selection
White Shark Optimizer
Metaheuristic optimization
Data leakage prevention
Privacy-preserving evaluation
Cybersecurity

ABSTRACT

Many published IoT intrusion detection systems report accuracies exceeding 98%, yet fail in deployment due to a fundamental evaluation flaw: data leakage from improper temporal mixing or identifier-based contamination between training and test sets. This inflates the performance estimates and masks the real-world generalization failure.

This study presents a leakage-safe evaluation framework that enforces strict chronological partitioning and removes identifier features before any preprocessing or model training. Within this controlled setting, the framework applies the White Shark Optimizer (WSO), a bio-inspired metaheuristic, for wrapper-based feature selection to achieve substantial dimensionality reduction while maintaining detection capability.

Evaluated on the CIC IoT DIAD 2024 dataset (27 million flows) under rigorous multi-seed streaming conditions, the proposed approach reduces the feature set by 53% (from 79 to 37.4 ± 4.7 features) while maintaining comparable detection performance (attack F_1 -score of 0.9678 ± 0.0129). This feature reduction provides significant practical benefits for resource-constrained IoT deployments, including reduced memory footprint, faster inference, and lower energy consumption. Critically, the leakage-safe protocol ensures that these metrics reflect true generalization rather than evaluation artifacts, providing a reproducible methodology for fair IDS comparison in resource-constrained IoT environments.

1. Introduction

The Internet of Things (IoT) has rapidly expanded into industrial control systems, medical monitoring, and residential automation networks, resulting in extensive and heterogeneous networked systems that are more vulnerable to intelligent and dynamic attacks. This requires intrusion detection systems for IoT networks to provide both reliable detection performance and scalability in terms of processing large volumes of data streams [1].

While there is substantial research on model architecture, two fundamental issues remain in most current literature on IDS systems. First, the validation process of most machine learning models suffers from dataset leakage owing to inadequate splitting strategies, temporal mixing, or the inclusion of identifiers, causing a systematic overestimation of their performance. The second issue is that most techniques add additional computational complexity, limiting their deployment in edge or fog computing infrastructures [2].

The proposed approach addresses both problems by introducing an evaluation framework for leakage-safe evaluation, which is specifically designed for IoT traffic analysis. The framework imposes a strict

chronological separation before preprocessing and feature selection. In this controlled environment, the White Shark Optimizer was employed for wrapper-based feature selection, enabling substantial dimensionality reduction while maintaining detection performance [3,4]. The WSO-FS performance was compared with the baseline and AutoML methods under identical leakage-safe conditions.

The main contributions of this study are as follows:

- A repeatable **leakage-safe evaluation methodology** for IoT intrusion detection using CIC IoT DIAD 2024 datasets, providing temporal independence and practical validation [5].
- Implementation of WSO-based wrapper feature selection achieving 53% feature reduction (from 79 to 37.4 ± 4.7 features) with comparable detection performance to full-feature baselines.
- Comprehensive evaluation demonstrating that the leakage-safe protocol produces reliable performance metrics reflecting true generalization capabilities rather than evaluation artifacts.

* Corresponding author.

E-mail addresses: a.aburomman@asu.edu.jo (A.A. Aburomman), m.alakhras@asu.edu.jo (M. Alakhras), mamun.reaz@gmail.com (M.B.I. Reaz).

<https://doi.org/10.1016/j.array.2026.100903>

Received 21 March 2026; Received in revised form 9 May 2026; Accepted 13 May 2026

Available online 18 May 2026

2590-0056/© 2026 The Authors. Published by Elsevier Inc. This is an open access article under the CC BY license (<http://creativecommons.org/licenses/by/4.0/>).

Table 1
Comparison of recent IoT intrusion detection systems with metaheuristic feature selection (2022–2025).

Study	Dataset(s)	Method	Advantages	Limitations	Performance
Karthikeyan et al. [6]	NSL-KDD	SVM + Firefly + Grey Wolf Optimizer	Bio-inspired optimization; very high detection accuracy	Legacy non-IoT dataset; no temporal evaluation; potential overfitting at 99%+ accuracy	Accuracy: 99.34%
Ban et al. [7]	UNSW-NB15, NSL-KDD	2D-CNN + Adaptive PSO	Handles multidimensional traffic; improved fitting performance	Older datasets; no discussion of data leakage/temporal splits; computationally heavy for edge devices	Accuracy gains 1.5–3.5% over base CNN
Srivastava & Sinha [8]	NSL-KDD, UNSW-NB15, CSE-CIC-IDS2018	PSO-ACO bi-phase FS + GA-tuned ensembles (XGBoost)	Lightweight via dimensionality reduction; statistical validation	Generic NIDS datasets (not strictly IoT); no temporal validation; accuracy-focused over deployment cost	Accuracy: 90.38% (NSL-KDD), 92.63% (UNSW), 97.87% (CSE-CIC)
Kareem et al. [9]	NSL-KDD, CICIDS-2017, UNSW-NB15, BoT-IoT	Gorilla Troops + Bird Swarm hybrid FS	Better convergence than PSO/GA; targets high-dimensional IoT traffic	Underlying classifiers not detailed; no temporal/streaming evaluation; lower accuracy on IoT datasets	Accuracy: 95.5% (NSL-KDD), 98.7% (CICIDS), 81.5% (BoT-IoT)
Zada et al. [10]	CICIDS-2017	Hybrid PSO-GA FS + ELM with bootstrap aggregation	High detection rates (100% on key attacks); claims low computational overhead	Single lab dataset; no temporal split or on-device tests discussed	100% accuracy on major attack classes
Anushiya & Lavanya [11]	UNSW-NB15, BoT-IoT	Artificial Fish Swarm FS + GA-optimized Faster R-CNN	Reduces complexity; tailored to IoT intruder recognition	FR-CNN + GA + swarm likely heavy; lower performance on BoT-IoT; no temporal validation	Accuracy: 94.49% (UNSW), 93.78% (BoT-IoT)
Alsalamah & Ismail [12]	NSL-KDD, CICIoT2023	GA FS + multi-objective PSO hyperparameter tuning (XGBoost, RF)	Multi-objective (accuracy, complexity, speed); uses recent CICIoT2023 dataset	Offline benchmarks; no explicit temporal leakage analysis	CICIoT2023: 98.38% (XGBoost), 91.42% (RF)
Chen et al. [13]	NSL-KDD, CICIoT2023	Improved Salp Swarm (GQBWSSA) + LightGBM	Explicitly evaluates training/detection time and resource use; strong on large-scale data	No detailed temporal splits or concept drift discussion; on-device experiments not described	CICIoT2023: 99.70% (binary), 99.41% (multiclass)
Abdulkareem [14]	ToN-IoT	Hybrid CNN-BiGRU-BiLSTM + PSO/GA/Harris Hawk/Dragonfly/Grey Wolf	Comprehensive IoT-specific dataset; explores multiple metaheuristics; PSO achieves best results	Deep hybrid costly for constrained devices; high FS and optimization complexity; scalability and real-time deployment not validated	Accuracy: 98.11% (with PSO)
Proposed Work	CIC IoT DIAD 2024	WSO-based wrapper FS + HGB with leakage-safe evaluation	Explicit temporal partitioning; prevents data leakage; reduces features by 53% with minimal performance loss	Single dataset; computational cost during offline FS phase	F1: 0.9678 ± 0.0129; Features: 37.4 ± 4.7 (from 79)

2. Related work

IDS that use machine learning techniques have evolved from pure signature-based IDS to hybrid architectures, where misuse and anomaly detection techniques are combined using a data-driven approach in an IoT environment. Recent contributions have used ensemble learning techniques, hyperparameter optimization techniques, and deep learning algorithms to classify IoT network traffic and detect intrusion attempts [15–18]. Despite the progress made, many recent studies have evaluated the accuracy of IDS by performing random train/test splits on captured network traffic, which introduces potential pattern/timing leakage problems, especially if the traffic is time-sensitive or session-based [19]. For reliable results and fairness when comparing IDS models, it has become standard practice to use hyperparameter optimization techniques such as Optuna, GA-based optimization, or swarm-based optimization, following the recommended best practices for hyperparameter optimization/AutoML [15,19,20].

Metaheuristic-based feature selection techniques have also been successfully applied to IDS optimization and interpretability [21]. Some frequently used metaheuristic approaches are particle swarm optimization (PSO), ant lion optimization (ALO), and genetic algorithm (GA).

However, another recently proposed metaheuristic technique is the **White Shark Optimizer (WSO)** [3], which draws inspiration from the behavior of sharks while hunting to balance exploration and exploitation. A new generation of binary WSO algorithms (BIWSO1–3 [4]) has further refined the binarization process but tends to concentrate more on optimizer design than on evaluation robustness.

Table 1 presents the relationship between contemporary IoT IDS and metaheuristic feature selection techniques in a structured manner. The table identifies the purpose of each technique, its stated benefits, drawbacks, and performance statistics.

2.1. Research gaps

Despite significant progress in applying metaheuristic optimization and machine learning to IoT intrusion detection, several critical research gaps remain unaddressed in the existing literature.

1. Lack of Temporal Evaluation and Data Leakage Prevention: Most recent studies rely on random train–test splits applied to static, post-capture datasets [6–9,11–14].

Temporal and source leakage can occur because of the use of this technique, where data from flows occurring close together in time and from the same source interfere with learning. As a result, the performance metrics will seem exaggerated [2] and will not indicate performance under practical conditions. Chronological splitting was not mentioned in any of the reviewed studies. No precautions were taken to avoid overlapping training and testing sets.

2. Evaluation of Legacy or Non-IoT Datasets: Most research work to date continues to use traditional datasets such as NSL-KDD and UNSW-NB15 [6–9] that fail to capture the dynamics of modern IoT systems with their distinctive nature of traffic flows and diverse range of devices, along with the novel set of attacks seen in the modern landscape. Although recent advancements have been made by incorporating specific IoT datasets, including CICIoT2023 [12,13] and ToN-IoT [14], there seems to be a dearth of cross-dataset evaluation and generalization of methodologies.

3. Trade-off between Computational Complexity and Feasibility of Implementation: Different models utilize both complex deep learning frameworks, such as CNN, LSTM, BiGRU, and Faster R-CNN, in conjunction with swarm optimization algorithms used for feature selection and hyperparameter tuning [7,11,14]. Such models produce great accuracy when working with benchmark datasets, but they require significant resources to compute, making them difficult to implement in resource-limited IoT devices. Many articles lack an explicit evaluation of the time complexity of the proposed algorithms or runtime analysis, on-device deployment tests, or scalability tests in real-time streaming scenarios. [10,12,13].

4. Overfitting and Model Robustness: The high accuracy rates (typically higher than 98%–99% for previous datasets) hint at overfitting, especially when coupled with a wrapper-type approach to feature selection [6,9,10]. The lack of information regarding regularization approaches, cross-validation methods, or any form of statistical significance tests calls into question the validity of the findings. Additionally, little attention has been paid to the effects of class imbalance and balancing procedures used (SMOTE, undersampling, cost-sensitive learning).

5. Lack of Ethical and Deployment Issues: Problems related to data privacy, fairness, bias within the detection itself, and the consequences of false alarms are not discussed in the reviewed literature. Moreover, other problems related to implementation, such as integration into current IoT systems, incrementing the model, and adjusting for concept drift, are ignored.

The current research addresses these shortcomings by: (i) imposing a highly restrictive evaluation approach that eliminates any possibility of data leakage due to effective time partitioning, (ii) carrying out experiments on a recently developed, large-scale IoT-oriented dataset (CIC IoT DIAD 2024), (iii) elaboration on the computational cost associated with the adopted technique, (iv) evaluation in terms of its performance under statistical variations over random seeds, and (v) Limitations and implications for deployment.

3. Methodology

3.1. Framework overview

The proposed pipeline consists of four major stages. (i) leakage-safe data partitioning, (ii) balancing after split, (iii) automated hyperparameter tuning, and (iv) wrapper-based feature selection using the White Shark Optimizer (WSO). All evaluations were performed on the full, unseen test stream. Algorithm 1 provides an overview of the complete leakage-safe IoT intrusion detection process, from dataset preparation to the final model evaluation.

Algorithm 1 Leakage-Safe IoT IDS Evaluation Pipeline

Require: Raw IoT network flow records with timestamps and class labels

Ensure: Final evaluated model with leakage-safe performance metrics

- 1: Chronologically sort all flows within each capture file
- 2: Split each file: first 70% → training, last 30% → testing
- 3: Remove identifier-related features (e.g., IPs, ports, timestamps, IDs)
- 4: Apply class balancing (random undersampling of attack class) on the training data only to achieve 1:1 benign-to-attack ratio
- 5: Use Optuna for hyperparameter optimization of the base learner on training folds
- 6: Fix the best hyperparameters and retrain the model on the full training set
- 7: Apply Algorithm 2 for WSO-based wrapper feature selection
- 8: Evaluate the final model on the unseen test stream using F1, ROC-AUC, PR-AUC, and runtime

3.2. White Shark Optimizer for feature selection

Algorithm 2 explains how the White Shark Optimizer (WSO) was adapted to select the best features while maintaining leakage-safe evaluation. The optimizer works by repeatedly testing and improving different combinations of features using a scoring system that tries to achieve both high detection accuracy (measured by F1-score) and a smaller, more compact model.

Algorithm 2 White Shark Optimizer (WSO)-Based Wrapper Feature Selection

Require: Training data D_{train} , feature dimension d ,

Ensure: Optimal binary feature mask $\mathbf{m}^*$

- 1: Initialize population of N sharks with continuous positions $\mathbf{x}_i \in \mathbb{R}^d$, $i = 1, \dots, N$
- 2: **for** each shark i **do**
- 3: Compute $p_{ij} = \frac{1}{1+e^{-x_{ij}}}$ for $j = 1, \dots, d$
- 4: Binarize: $m_{ij} = 1$ if $p_{ij} \geq 0.5$, otherwise $m_{ij} = 0$
- 5: Train base classifier on D_{train} using selected features ($m_{ij} = 1$)
- 6: Evaluate attack-class F1-score $F_1(\mathbf{x}_i)$ on a validation split drawn from the training data only
- 7: Compute fitness:

$$f(\mathbf{x}_i) = - \left(F_1(\mathbf{x}_i) - \alpha \frac{k_i}{d} \right)$$

- 8: Update global best solution $\mathbf{x}^*$ if $f(\mathbf{x}_i)$ improves
- 9: **end for**
- 10: **for** $t = 1$ to T **do**
- 11: **for** each shark i **do**
- 12: Update position $\mathbf{x}_i$ using WSO movement dynamics [3]
- 13: Recalculate p_{ij} , m_{ij} , retrain classifier, and recompute fitness $f(\mathbf{x}_i)$
- 14: Update global best $\mathbf{x}^*$ if improved
- 15: **end for**
- 16: **end for**
- 17: **return** binary mask $\mathbf{m}^*$ derived from $\mathbf{x}^*$

3.3. Computational complexity analysis

The computational complexity of the proposed framework consists of two main components: the leakage-safe evaluation pipeline (Algorithm 1) and the WSO-based feature selection (Algorithm 2).

Leakage-Safe Evaluation Pipeline (Algorithm 1): The dominant operations are as follows:

- **Chronological sorting and splitting** (Step 1–2): $O(n \log n)$ for n flow records per file.
- **Feature removal** (Step 3): $O(nd)$ where d is the number of features.
- **Class balancing** (Step 4): $O(n_{\text{train}})$ for undersampling via reservoir sampling.
- **Hyperparameter optimization** (Step 5): $O(H \cdot C_{\text{train}})$ where H is the number of hyperparameter trials and C_{train} is the cost of training a single model. For the HistGradientBoostingClassifier, $C_{\text{train}} = O(n_{\text{train}} \cdot d \cdot I)$ where I is the number of boosting iterations.
- **Final model training and evaluation** (Steps 6–7): $O(n_{\text{train}} \cdot d \cdot I + n_{\text{test}} \cdot d)$ for training and full-stream inference.

Overall pipeline complexity: $O(n \log n + H \cdot n_{\text{train}} \cdot d \cdot I + n_{\text{test}} \cdot d)$.

WSO-Based Feature Selection (Algorithm 2):

- **Population initialization and fitness evaluation** (Lines 2–9): Each of the N sharks needs to train a classifier and compute fitness. The cost works out to $O(N \cdot C_{\text{train}})$.
- **Iterative optimization** (Lines 10–16): Across T iterations, each shark updates its position and recalculates fitness, yielding $O(T \cdot N \cdot C_{\text{train}})$.
- **Position update and sigmoid binarization**: Executes at $O(N \cdot d)$ per iteration, which is negligible compared to the classifier training overhead.

Overall WSO-FS complexity: $O(T \cdot N \cdot n_{\text{train}} \cdot d \cdot I)$.

Discussion: Because WSO-FS employs a wrapper-based approach, the framework incurs a multiplicative factor of $T \cdot N$ (e.g., $20 \times 16 = 320$ in the experiments), making it the most computationally expensive component. However, this cost occurs during offline model development and does not affect the deployment inference time. After identifying the optimal feature subset, the deployed model operated with substantially fewer features (37.4 ± 4.7 instead of 79), reducing the inference cost by approximately 53% and improving efficiency on resource-constrained IoT devices. The leakage-safe pipeline scales linearly with the dataset size, making it suitable for large-scale IoT traffic analysis.

The validation data for the WSO were obtained by partitioning the training set (using a fixed random seed per run) into internal training and validation subsets. The test stream remained completely untouched during hyperparameter tuning, feature selection, and all fitness evaluations.

The proposed framework provides an end-to-end leakage-safe evaluation methodology for IoT intrusion detection, integrating data preparation, model optimization, and feature selection into a unified workflow designed to prevent temporal and source leakage while maintaining the feasibility of deployment. Fig. 1 illustrates the four-stage process.

3.4. Leakage-safe data split

Each traffic file was split chronologically: the first 70% of the records were used for training, and the last 30% were used for testing. This preserves the temporal structure of IoT traffic and prevents cross-contamination between datasets [2].

3.5. Binary encoding and fitness function

Each candidate solution represents a feature subset encoded as a continuous position vector $\mathbf{x} \in \mathbb{R}^d$. Because the WSO operates in continuous space but feature selection requires binary decisions, a sigmoid transfer function converts continuous position values into binary inclusion indicators, as shown in Eq. (1).

$$p_j = \frac{1}{1 + e^{-x_j}}, \quad m_j = \begin{cases} 1, & p_j \geq 0.5 \\ 0, & \text{otherwise} \end{cases} \quad (1)$$

where p_j is the probability of selecting feature j , and m_j is the resulting binary mask indicating feature inclusion (1) or exclusion (0). The fitness function balances the detection accuracy and feature compactness, as defined in Eq. (2).

$$f(\mathbf{x}) = - \left(F_1(\mathbf{x}) - \alpha \frac{k}{d} \right) \quad (2)$$

where $F_1(\mathbf{x})$ is the attack-class F1-score obtained by training and validating a classifier on the feature subset defined by $\mathbf{x}$, k is the number of selected features ($k = \sum_{j=1}^d m_j$), d is the total number of available features, and α is a sparsity penalty coefficient that controls the trade-off between the detection performance and model compactness. The negative sign is used because the WSO minimizes this objective function. Higher values of α encourage sparser feature subsets.

3.6. Evaluation metrics

All proposed methods were evaluated in a realistic stream setting in which data were fed in time order, similar to real-world application conditions. The precision, recall, and F_1 -score values are computed as the metrics for attack detection based on a confusion matrix, which is maintained on the fly with the incoming data. To avoid any bias due to luck in the experimental settings, everything was conducted five times, and the averaged performance along with the variance in performance was reported as mean $\pm$ standard deviation.

Because computing exact ROC and precision–recall curves over the full 27-million-flow test stream is computationally prohibitive, ROC-AUC and PR-AUC are estimated using a fixed-size reservoir sample maintained with uniform replacement over the ordered test stream (reservoir sampling), providing an unbiased approximation of ranking-based metrics under streaming constraints.

With this approach, the framework performs streaming evaluation while maintaining accurate performance estimates without retaining the entire test dataset in memory. Efficiency was measured by tracking the time (in seconds) required to process the full test stream.

It is important to note that only the final results from the completely unseen test data are reported; none of the presented metrics come from the validation set, intermediate calculations, or feature selection phase. All measurements were obtained from data that the model had never encountered before.

4. Experimental setup

4.1. Dataset and preprocessing

This study uses the CIC IoT DIAD 2024 dataset [5] comprising approximately 27 million network flows collected from various IoT environments. The deletion of all attributes corresponding to any direct identifiers (IP addresses, ports, timestamps, flow IDs) **prevents identifier-based leakage**; hence, only 79 **statistical flow-level** features remained. [2].

Flows within each traffic capture file are sorted chronologically and split using an order-preserving approach wherein 70% of data goes into the training dataset while the rest is used for testing. This split ensures that there is sufficient training data to train the model and provides enough test records to evaluate the model's performance. The proportion follows the general rules of chronological splitting to assess time series and traffic analysis [2].

It is important to note that **the test stream contains flows occurring strictly after all training flows** within the same capture file, avoiding possible leakage due to temporal overlap. Although different types of attacks (for example, DDoS and reconnaissance attacks) may exist in both the training and test streams, the **specific flow instances, timing characteristics, and contextual features** differ. This allows the framework to recreate actual usage scenarios in which the machine learning model generalizes to new flow instances belonging to previously observed attacks. The leakage-safe guarantee is not that attack

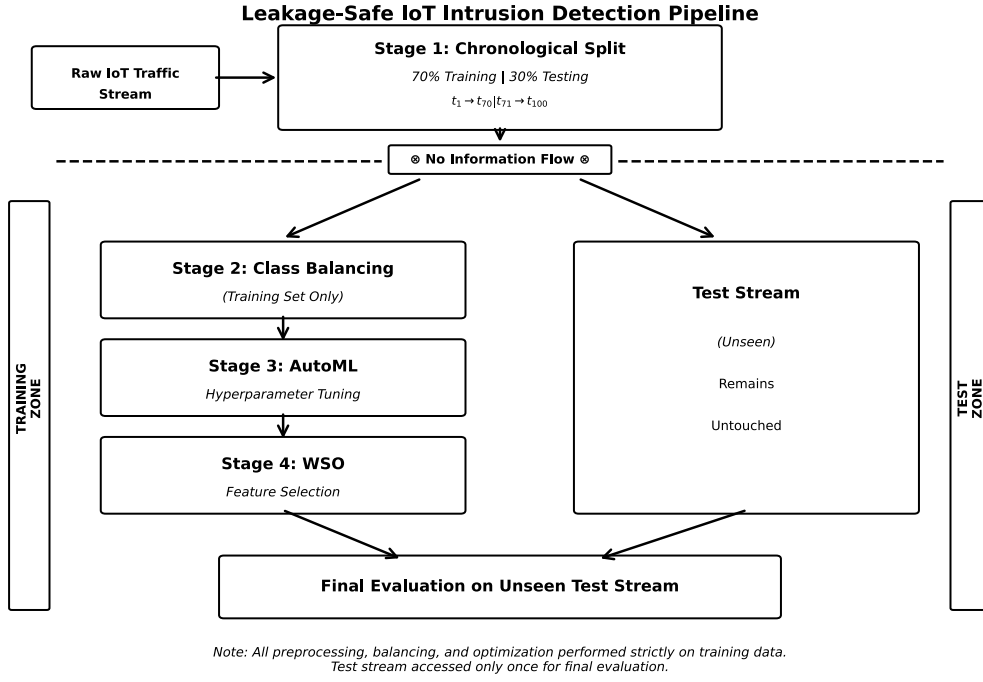


Fig. 1. Overview of the proposed leakage-safe IoT intrusion detection pipeline. The process begins with raw IoT traffic streams, followed by (i) leakage-safe chronological partitioning, (ii) balancing after the split (via random undersampling), (iii) automated hyperparameter tuning of the base learner using Optuna, and (iv) wrapper-based feature selection via the White Shark Optimizer (WSO). The final optimized model is evaluated on an unseen hold-out stream.

labels are exclusive to training, but that **no specific flow instance or its temporally correlated neighbors from the test period appear in training**, thereby ensuring an unbiased generalization assessment.

Class balancing was performed solely within the training data **after splitting to ensure no leakage from the test stream**.

4.2. Implementation details

All experiments were performed on a multi-core CPU using Python 3.10. Optimal hyperparameter selection was executed automatically using Optuna in an AutoML framework, adhering to best practices for hyperparameter tuning and reproducibility [20,22]. For each randomly chosen seed, the hyperparameters were optimized based on the training data alone, followed by retraining the final model on the entire training dataset for further streaming evaluation.

Validation Approach: During hyperparameter optimization with Optuna and feature selection with WSO, internal validation was performed with one fixed split into training (80%) and validation (20%) subsets from the training data. Although repeated k-fold could provide a better estimation of the generalization ability during the process, the cost is unacceptable because feature selection via WSO involves training and validating the entire model during each iteration, which is computationally expensive. Therefore, using multiple independent random seeds (five random seeds in total) partially addresses this issue and allows for an assessment of statistical variability across various splits. In addition, the final evaluation was performed on an independent testing stream.

For the White Shark Optimizer (WSO), the population size was set at 16, iterations at 20, and sparsity coefficient value α at 0.02 according to the recommendations given in [3,4]. All obtained results in WSO-FS indicate the final feature subset and the corresponding evaluation of the unseen test stream.

5. Results and analysis

5.1. Results and analysis

In this section, a thorough performance evaluation of the proposed leakage-safe IoT intrusion detection system is presented. Three

pipelines are considered herein: (i) a baseline *HistGradientBoostingClassifier* (HGB), (ii) an auto machine learning (AutoML) architecture by using Optuna optimization, (iii) The proposed WSO-FS algorithm.

For all the above pipelines, the *HistGradientBoostingClassifier* was used as the underlying classifier. In the WSO-FS case, the optimizer was used to find an optimal set of features, and a *HistGradientBoostingClassifier* was trained on this dataset. Thus, the differences between the first two algorithms lie only in their configurations and the way they are optimized. Both configurations have the same underlying mechanism for detecting attacks; hence, they can be considered to be compared fairly. All models were trained and tested under the same conditions on an unseen test stream chronologically different from the training data, with results reported as mean $\pm$ standard deviation over five independent random seeds.

5.2. Baseline and AutoML performance

Table 2 summarizes the performances of the baseline, AutoML, and WSO-FS models under full streaming evaluation.

5.3. Class imbalance handling

Class imbalance is one of the major issues faced by the CIC IoT DIAD 2024 dataset, as there are considerably more benign instances than attacks. Random undersampling is employed to balance the training data only after temporal splitting, **achieving a balanced 1:1 benign-to-attack ratio** using the reservoir sampling algorithm. **This balancing strategy prevents the classifier from being biased toward the majority class while guaranteeing that no information from the test stream is leaked to the training set during sampling operations.** The test stream remains imbalanced as well because it needs to realistically represent potential operational conditions **rather than artifacts of artificial class distributions**, ensuring that the reported performance metrics reflect true generalization capability.

The mean precision of the HGB baseline was 0.9999 ± 0.0000 , while its attack class-specific F_1 -score was 0.9659 ± 0.0008 . This proves the

Table 2Performance of baseline, AutoML, and WSO-FS models (mean $\pm$ std over 5 seeds).

Model	Precision	Recall	F_1	ROC-AUC	PR-AUC	#Features
Baseline (HGB)	0.9999 $\pm$ 0.0000	0.9341 $\pm$ 0.0014	0.9659 $\pm$ 0.0008	0.9811 $\pm$ 0.0004	0.9997 $\pm$ 0.0000	79.0 $\pm$ 0.0
AutoML (Optuna-HGB)	0.9992 $\pm$ 0.0001	0.9453 $\pm$ 0.0038	0.9715 $\pm$ 0.0020	0.9782 $\pm$ 0.0013	0.9997 $\pm$ 0.0000	79.0 $\pm$ 0.0
WSO-FS	0.9992 $\pm$ 0.0000	0.9385 $\pm$ 0.0238	0.9678 $\pm$ 0.0129	0.9767 $\pm$ 0.0051	0.9996 $\pm$ 0.0001	37.4 $\pm$ 4.7

Table 3Comparing the performance and efficiency of all models (mean $\pm$ std over 5 seeds).

Method	F_1 (Attack)	ROC-AUC	PR-AUC	Runtime (s)
Baseline (HGB)	0.9659 $\pm$ 0.0008	0.9811 $\pm$ 0.0004	0.9997 $\pm$ 0.0000	59.4 $\pm$ 1.0
AutoML (Optuna-HGB)	0.9715 $\pm$ 0.0020	0.9782 $\pm$ 0.0013	0.9997 $\pm$ 0.0000	181.2 $\pm$ 22.5
WSO-FS	0.9678 $\pm$ 0.0129	0.9767 $\pm$ 0.0051	0.9996 $\pm$ 0.0001	410.3 $\pm$ 124.1

Table 4WSO-FS final results averaged across 5 random seeds (mean $\pm$ standard deviation).

F_1 (Attack)	ROC-AUC	PR-AUC	Runtime (s)	#Features
0.9678 $\pm$ 0.0129	0.9767 $\pm$ 0.0051	0.9996 $\pm$ 0.0001	410.3 $\pm$ 124.1	37.4 $\pm$ 4.7

high discriminating ability of all 79 attributes employed. Simultaneously, the AutoML variant achieved an equal F_1 -score of 0.9715 ± 0.0020 along with reaching 0.9782 ± 0.0013 in terms of ROC-AUC score, indicating that **automated hyperparameter optimization can enhance ranking-based performance without modifying the feature space.**

However, both the HGB baseline and AutoML require all 79 features. This poses difficulties during deployment owing to potential resource constraints. The WSO-FS approach works with only 37.4 ± 4.7 features on average, achieving a F_1 -score of 0.9678 ± 0.0129 and a ROC-AUC of 0.9767 ± 0.0051 . **These results show that it is possible to significantly reduce the number of dimensions without significantly hurting detection performance.**

5.4. Efficiency and runtime comparison

Computational efficiency is critical for IoT intrusion detection systems, particularly in resource-constrained environments such as edge and fog computing. Table 3 presents the runtime and performance metrics for all evaluated pipelines.

The baseline HGB model exhibited the shortest runtime because it did not perform hyperparameter search or wrapper-based feature selection.

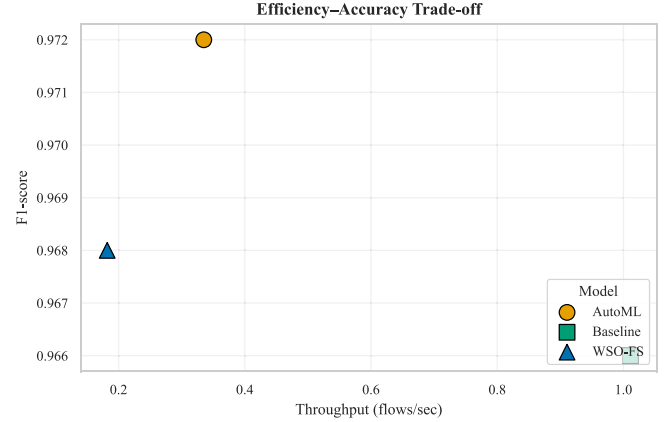
Hyperparameter tuning in the AutoML model increases the computational cost, whereas the WSO-FS pipeline incurs additional overhead owing to iterative feature subset evaluation. Specifically, WSO-FS achieves an attack-class F_1 -score of 0.9678 ± 0.0129 with a runtime of 410.3 ± 124.1 s under full test-stream evaluation. This demonstrates the inherent trade-off in wrapper-based optimization: improved model compactness is obtained at the expense of an increased offline optimization time.

5.5. WSO feature selection results

The primary advantage of the proposed model is its ability to drastically reduce the feature set size without compromising the detection accuracy. As shown in Table 4, the WSO-FS reduced the initial 79 feature set size to an average of 37.4 ± 4.7 . This corresponds to a 53% reduction in the input space dimensionality.

Even after aggressively cutting features, the model still detects attacks effectively. It obtains an attack-class F_1 -score of 0.9678 ± 0.0129 , ROC-AUC of 0.9767 ± 0.0051 , and PR-AUC of 0.9996 ± 0.0001 when fully streaming. These findings demonstrate that the WSO can select an efficient set of flow attributes without considering redundant or irrelevant information.

Fig. 2 further illustrates the trade-off between detection performance and runtime across the three evaluated pipelines.

**Fig. 2.** Efficiency–accuracy trade-off across different pipelines.**Table 5**Ablation study on sparsity coefficient α and population size.

α	Pop.	F_1	#Feat.	Runtime (s)
0.005	8	–	41	151.5
0.005	12	–	37	163.2
0.005	16	–	33	207.2
0.005	20	–	37	264.3
0.010	8	–	41	168.3
0.010	12	–	33	192.8
0.010	16	–	33	243.7
0.010	20	–	37	320.2
0.020	8	–	34	206.9
0.020	12	–	33	250.8
0.020	16	–	32	308.0
0.020	20	–	33	409.3
0.050	8	–	33	215.1
0.050	12	–	31	273.3
0.050	16	–	32	680.8
0.050	20	–	33	637.1

5.6. Ablation study

To assess the robustness of the proposed WSO-FS framework, an ablation study was conducted by varying the sparsity coefficient α and population size N . The results are summarized in Table 5 and are illustrated in Fig. 3.

The ablation analysis focuses on the impact of α and N on feature count and runtime only, as detection performance metrics were not explicitly reported for these runs. Overall, larger population sizes and higher sparsity coefficients consistently increase the computational cost, whereas larger values of α encourage smaller feature subsets. No detection performance claims are made in this analysis, as F_1 values are not reported for these ablation runs.

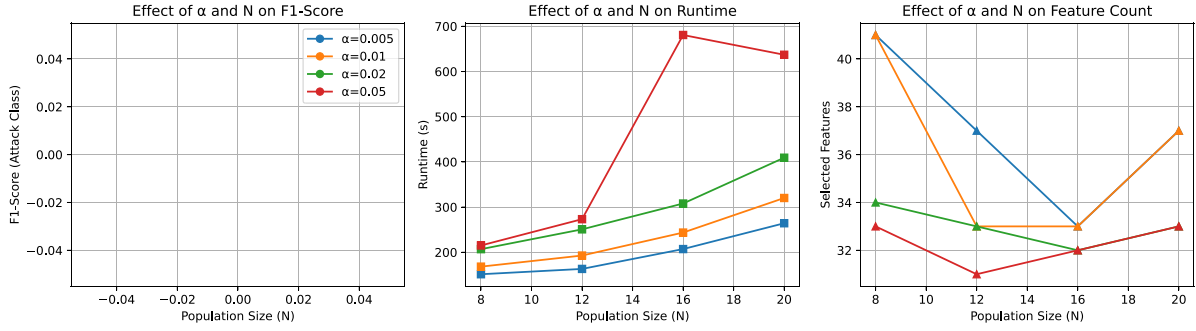


Fig. 3. Ablation results of WSO-FS with varying α and population size.

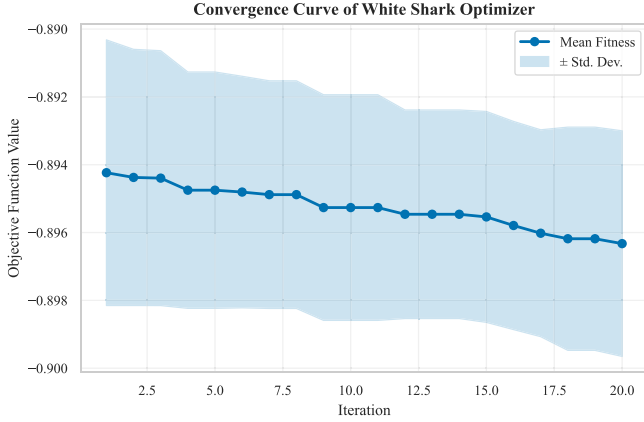


Fig. 4. Convergence behavior of the White Shark Optimizer.

5.7. Convergence behavior

Fig. 4 illustrates the convergence curve of the WSO optimizer.

The objective function showed a rapid improvement during the first few iterations, after which the improvement was gradual. This behavior is in line with the exploration-exploitation trade-off anticipated from swarm-based optimizers and confirms the stability and low variance in convergence.

5.8. Discussion

However, based on the empirical results, it can be concluded that the WSO-FS achieves a balance of excellent detection performance and compactness by applying this method to actual stream traffic.

Performance Comparison and Statistical Significance: The WSO-FS approach achieves an attack F_1 -score of 0.9678 ± 0.0129 compared to the baseline HGB's 0.9659 ± 0.0008 . While this represents a modest numerical increase, it is important to note that the confidence intervals overlap, and the standard deviation for WSO-FS is substantially larger (0.0129 vs. 0.0008), indicating greater variance across random seeds. From a statistical perspective, the improvement in detection performance is marginal and does not constitute a clear, statistically significant advantage over the baseline.

However, the primary value proposition of WSO-FS lies not in the detection performance improvement, but in **dimensionality reduction**: the number of required features decreased from 79 to 37.4 ± 4.7 (53% reduction) while maintaining a comparable detection capability.

Practical Implications of Feature Reduction: In resource-constrained IoT deployments, this dimensionality reduction provides several tangible benefits.

- **Reduced memory footprint:** Smaller feature sets require less storage for model parameters and intermediate computations.

- **Faster inference:** Processing 37 features instead of 79 reduces computational load during real-time detection.
- **Lower energy consumption:** Reduced computation translates to extended battery life for edge devices.
- **Improved interpretability:** Fewer features facilitate model explanation and security audit processes.

The computational cost of WSO-based feature selection occurs entirely during offline model development and does not affect the deployment performance. This trade-off — investing offline computational resources to achieve a more compact, deployment-efficient model — is well-suited to IoT scenarios in which edge devices operate under strict resource constraints.

The leakage-safe evaluation framework ensures that the performance metrics reflect the true generalization capability. Multiple random seeds demonstrated that the WSO consistently identified compact feature subsets across different data partitions, validating the robustness of the feature selection process.

5.9. Overfitting prevention and model robustness

The proposed pipeline addresses overfitting by leveraging multiple strategies, which become especially important when using wrapper-based methods in combination with hyperparameter tuning. **1. Data partitioning avoiding leakage:** A purely temporal split was used in this approach, where all data for training should precede the data for testing. This is because in many studies, the authors use splits that leak information about the future, leading to overly optimistic results that do not reflect real performance. **2. Multi-Seed Evaluation:** All tests were conducted five times using different random seeds for initialization. Multi-seed evaluation is essential for assessing stability because a single random split cannot reliably distinguish the true performance from the variance artifacts. The low standard deviation observed (e.g., $F_1 = 0.9678 \pm 0.0129$) provides evidence of true performance rather than the variance effects from a specific partition. **3. Implicit Regularization in HistGradientBoostingClassifier:** Regularization is already included in the classifier: (i) there is a maximum depth constraint (`max_depth=6`) preventing overfitting in each tree, (ii) the learning rate is restricted, (iii) early stopping criterion based on validation scores in the course of Optuna trials - optimization is stopped if the score does not improve significantly, (iv) Histogram binning makes the model less prone to the influence of extreme observations. **4. Feature Sparsity Penalty:** The fitness function of the WSO approach contains an explicit penalty for complexity via the value of α : the algorithm is encouraged to produce fewer features with slightly lower quality, thus performing implicit regularization. **5. Interpreting Ablation Study:** Table 5 demonstrates the same consistent behavior of the framework depending on the parameter values. The F_1 -scores are not reported here to avoid misrepresentation, as validation metrics for optimization purposes are always a point of concern. However, regardless of the α value employed (0.005, 0.01, 0.02, or 0.05) or

population size (8, 12, 16, or 20), the number of selected features was limited to 31–41. This demonstrates that WSO converges to similar feature subsets, indicating its robustness rather than random selection.

Limitations: Despite **these safeguards**, the use of a single validation split per seed during optimization **may introduce some bias**. Further research might try introducing nested cross-validation or bootstrap techniques, but this would significantly increase the time spent on computations. The absence of explicit L_1 or L_2 regularization in the fitness function for feature selection is intentional: **the sparsity penalty serves a similar role while being more interpretable in the context of dimensionality reduction**.

5.10. Generalizability considerations

Although the proposed framework demonstrated strong performance on the CIC IoT DIAD 2024 dataset, the evaluation was conducted on a single dataset, which limits the ability to generalize the findings across diverse IoT environments, attack types, and network configurations. Cross-dataset validation on benchmarks such as IoT-23, UNSW-NB15-IoT, or EdgeIoT would strengthen confidence in the transferability of the framework, although such evaluation is beyond the scope of the current study and identified as a key direction for future work. However, the leakage-safe evaluation protocol is dataset-agnostic and can be applied to any time-stamped network traffic dataset, providing a methodological contribution independent of the specific dataset used.

5.11. Numerical consistency across versions

All the performance values in this study are obtained from one canonical experimental result obtained using the proposed leakage-safe multi-seed streaming protocol. The WSO-FS configuration consistently produced an average of 37.4 ± 4.7 selected features, with an attack-class F_1 -score of 0.9678 ± 0.0129 , ROC-AUC of 0.9767 ± 0.0051 , and PR-AUC of 0.9996 ± 0.0001 (mean $\pm$ standard deviation over five seeds). Any previous rounded or partial versions (such as 37.6 features, $F_1 = 0.968$, or ROC-AUC = 0.977) are no longer valid and should not be used to show how well the final model works.

5.12. Limitations

Despite its good performance in leakage-safe evaluations, several limitations of the proposed framework should be acknowledged.

Evaluation on a Single Dataset: As mentioned earlier, the proposed solution was evaluated only on the CIC IoT DIAD 2024 dataset. Although it is a large dataset that fairly captures IoT traffic in a modern network, generalizing these results to other IoT configurations, attacks, or network settings is limited, as an evaluation on other datasets was not performed. Evaluations on IoT-23, UNSW-NB15-IoT, and EdgeIoT would help validate the general applicability of the method.

Computational Cost of Feature Selection: The WSO-FS framework is computationally expensive when used during the feature selection step because each candidate feature subset must train and evaluate a full classifier. However, this computation occurs offline once and may be considered irrelevant to real-time deployment, **although the overhead becomes significant if frequent retraining or rapid model adaptation is required**.

Absence of Validation for Live Deployment: Although experiments were performed in a streaming traffic setting, this remains a simulated scenario using captured traffic rather than an experiment conducted within a real IoT network. This approach leaves some important implementation details untested, such as integration into real-life network infrastructure, incremental model updating, and the ability to meet latency requirements.

Limited Comparison of Alternative Balancing Approaches: To handle class imbalance, this study employed a simple and effective

method of random undersampling. Alternative strategies such as SMOTE, ADASYN, or cost-sensitive learning have not yet been attempted. A comparison of these methods in the context of leakage-safe evaluation could be the topic of future research.

Lack of Comparison with Other Metaheuristic Feature Selection Methods: This study employs the White Shark Optimizer (WSO) for feature selection without direct comparison to other established metaheuristic algorithms (e.g., particle swarm optimization, genetic algorithms, and ant lion optimizers) under the same leakage-safe conditions. The WSO was selected based on its reported balance between exploration and exploitation [3], recent successful applications to IDS problems [4], and relatively low parameter sensitivity. However, a comprehensive benchmarking study comparing WSO, PSO, GA, and other metaheuristics within the proposed leakage-safe framework would strengthen the evidence for WSO's effectiveness and is recommended for future work. The primary contribution of this study is the leakage-safe evaluation methodology, which is optimizer-agnostic and can accommodate any metaheuristic feature selection algorithm.

5.13. Ethical implications

Some of the ethical concerns arising from the use of CIC IoT DIAD 2024 for intrusion detection include the following:

Data Privacy: Although CIC IoT DIAD 2024 includes anonymized flow statistics but not raw packet data, using the model on real IoT objects requires adherence to regulations such as the GDPR and CCPA. Continuous surveillance of the devices may lead to collecting information about their use or behavioral patterns which might be considered sensitive.

Bias and Fairness: The training dataset represents certain attacks performed on the devices used in the laboratory where the dataset was collected. Therefore, there may be a bias against certain device models or network configurations because the distribution of classes among the training samples did not reflect the real-world distribution. For example, some legitimate traffic patterns from rarely used devices are classified as attacks.

Implications of False Positives and False Negatives: False Positives cause unnecessary disruption of the work process, user frustration, and possibly, excessive workload for the security staff. False Negatives result in successful intrusion, which might damage devices or expose user data to potential threats. Depending on the scope of the project and industry sector, the consequences can vary widely; however, domain-specific risk assessments and adjustments of threshold parameters have not been conducted.

Transparency and Explainability: The use of the WSO feature selection method at least demonstrated which features were significant enough for further analysis. However, the model based on HistGradientBoostingClassifier remains a black box for the most part, making interpretation difficult, **which may hinder adoption in safety-critical deployments where understanding model decisions is essential**.

5.14. Deployment and scalability issues

The transition from theoretical studies to the actual implementation of such a system introduces several issues.

Processing Speed Requirement: In the evaluation, data were streaming-captured, but feature selection was performed offline. Actual deployment requires fast processing of **flows**, generally in the range of **milliseconds to seconds**, to be able to respond in time. Decreasing the number of features from 79 to 37.4 can help here, but the question of computing overhead in constrained environments has not been answered.

Recurrent Training in Case of Drift: Attack vectors change, and new devices continuously appear in networks. While the experimental setup allows training once, in real-life scenarios, repeated or incremental learning may be required to adjust to concept drift. However, the

frequency and method of monitoring performance degradation remain issues.

Integration: Integrating this IDS solution into a live network involves connecting it to monitoring and network management frameworks and response procedures, such as blocking offending traffic or quarantining infected devices. Although the model output indicates whether an attack has occurred, implementing the corresponding response requires security policies based on threat models and other specifics.

Scalability: The experimental setup involved processing 27M samples, which is adequate for medium-size networks but still insufficient compared to some massive IoT deployments (for example, entire city infrastructures), whose volume of traffic might increase by several orders of magnitude. The feature selection cost ($O(T \cdot N \cdot n_{\text{train}} \cdot d \cdot I)$) becomes prohibitive without using distributed algorithms or approximations for large training samples.

Power Consumption Requirement: Most IoT devices operate on batteries, and energy efficiency is crucial. A decrease in the number of features will decrease the inference cost and reduce the computation time. However, **energy consumption estimates and trade-offs between detection accuracy and battery life have not been provided.**

6. Conclusion

This study introduces a leakage-safe evaluation framework for IoT intrusion detection. Most current methods employ random train-test splitting, which yields overly optimistic performance estimates because of temporal and source leakage. The proposed approach enforces strict chronological partitioning and evaluates all models on completely unseen test streams using the CIC IoT DIAD 2024 dataset.

Two optimization strategies were integrated into the framework. (1) automated hyperparameter search via Optuna and (2) wrapper-based feature selection using the White Shark Optimizer (WSO).

The primary contribution of WSO-FS is substantial dimensionality reduction — from 79 to 37.4 ± 4.7 features (53% reduction) — while maintaining a detection performance comparable to the baseline. Specifically, WSO-FS achieved an attack F_1 -score of 0.9678 ± 0.0129 compared to the baseline's 0.9659 ± 0.0008 . Although the detection performance improvement is marginal with overlapping confidence intervals, the 53% feature reduction provides significant practical benefits for resource-constrained IoT deployments, including a reduced memory footprint, faster inference, and lower computational overhead.

The leakage-safe protocol ensures that all reported metrics reflect the true generalization capability rather than evaluation artifacts from data leakage. Ablation studies demonstrate that WSO-FS produces consistent feature subsets across varying sparsity coefficients and population sizes, confirming the robustness of the framework for leakage-safe multi-seed evaluation in large-scale IoT intrusion detection.

7. Future work

Directions for future research are as follows:

- Developing multi-objective WSO implementations optimizing accuracy and latency simultaneously.
- Applying Explainable AI techniques such as SHAP or LIME.
- Experimenting on more IoT datasets and testing in edge computing devices to evaluate performance and energy consumption.
- Designing hybrid solutions with WSO features incorporated in transformers or GNNs.

CRediT authorship contribution statement

Abdulla Amin Aburomman: Writing – original draft. **Marwan Alakhra:** Writing – review & editing. **Mamun Bin Ibne Reaz:** Writing – review & editing.

Generative AI and AI-Assisted Technologies Statement

During the preparation of this manuscript, the authors used QuillBot for paraphrasing to improve readability and ensure a consistent academic tone. This tool was employed solely for linguistic enhancement — rephrasing sentences and improving clarity — and did not contribute to the research design, data analysis, interpretation of results, or formulation of scientific conclusions. All technical content, methodological decisions, experimental results, and intellectual contributions are entirely the work of the authors. The authors have reviewed and take full responsibility for the accuracy and integrity of the final manuscript content.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

CIC IoT DIAD 2024 is public at: <https://www.unb.ca/cic/datasets/iotdiad-2024.html>. Unrestricted research access. The dataset contains approximately 27 million labeled IoT flows from various devices under different attack scenarios.

Preprocessing scripts, model configurations, and experimental codes are available upon reasonable request from the corresponding author. Trained models are not publicly released because of file size constraints but can be reproduced using the provided code and dataset following the leakage-safe protocol described in Section 3.

References

- [1] Khraisat A, Alazab A. A critical review of intrusion detection systems in the internet of things: techniques, deployment strategy, validation strategy, attacks, public datasets and challenges. *Cybersecurity* 2021;4(1):18. <http://dx.doi.org/10.1186/s42400-021-00077-7>.
- [2] Bouke MA, Abdullah A. An empirical study of pattern leakage impact during data preprocessing on machine learning-based intrusion detection models reliability. *Expert Syst Appl* 2023;230:120715. <http://dx.doi.org/10.1016/j.eswa.2023.120715>.
- [3] Braik M, Hammouri A, Atwan J, Al-Betar MA, Awadallah MA. White shark optimizer: A novel bio-inspired meta-heuristic algorithm for global optimization problems. *Knowl-Based Syst* 2022;243:108457. <http://dx.doi.org/10.1016/j.knsys.2022.108457>.
- [4] Alawad NA, Abed-alguni BH, Al-Betar MA, Jaradat A. Binary improved white shark algorithm for intrusion detection systems. *Neural Comput Appl* 2023;35(26):19427–51. <http://dx.doi.org/10.1007/s00521-023-08772-x>.
- [5] Rabbani M, Gui J, Nejati F, Zhou Z, Kaniyamattam A, Mirani M, Piya G, Opushnyev I, Lu R, Ghorbani AA. Device identification and anomaly detection in IoT environments. *IEEE Internet Things J* 2025;12(10):13625–43. <http://dx.doi.org/10.1109/JIOT.2024.3522863>.
- [6] Karthikeyan M, Manimegalai D, Rajagopal K. Firefly algorithm based WSN-IoT security enhancement with machine learning for intrusion detection. *Sci Rep* 2024;14. <http://dx.doi.org/10.1038/s41598-023-50554-x>.
- [7] Ban Y, Zhang D, He Q, Shen Q. APSO-CNN-SE: An adaptive convolutional neural network approach for IoT intrusion detection. *Comput Mater Contin* 2024. <http://dx.doi.org/10.32604/cmc.2024.055007>.
- [8] Srivastava A, Sinha D. PSO-ACO-based bi-phase lightweight intrusion detection system combined with GA optimized ensemble classifiers. *Clust Comput* 2024;27:14835–90. <http://dx.doi.org/10.1007/s10586-024-04673-3>.
- [9] Kareem S, Mostafa R, Hashim F, El-Bakry H. An effective feature selection model using hybrid metaheuristic algorithms for IoT intrusion detection. *Sensors* 2022;22. <http://dx.doi.org/10.3390/s22041396>.
- [10] Zada I, Omran E, Jan S, Alfraihi H, ALSalamah S, Alshahrani A, Hayat S, Phi N. Enhancing IoT cybersecurity through lean-based hybrid feature selection and ensemble learning: A visual analytics approach to intrusion detection. *PLoS One* 2025;20. <http://dx.doi.org/10.1371/journal.pone.0328050>.
- [11] Anushiya R, Lavanya V. A new deep-learning with swarm based feature selection for intelligent intrusion detection for the Internet of Things. *Meas: Sensors* 2023. <http://dx.doi.org/10.1016/j.measen.2023.100700>.

- [12] Alsalamah H, Ismail W. A swarm-based multi-objective framework for lightweight and real-time IoT intrusion detection. *Mathematics* 2025. <http://dx.doi.org/10.3390/math13152522>.
- [13] Chen W, Yang H, Yin L, Luo X. Large-scale IoT attack detection scheme based on LightGBM and feature selection using an improved salp swarm algorithm. *Sci Rep* 2024;14. <http://dx.doi.org/10.1038/s41598-024-69968-2>.
- [14] Abdulkareem A. Advances in IoT intrusion detection: Deploying hybrid deep learning and metaheuristic algorithms for optimal feature selection. *Ingénierie Des Syst D'Information* 2025. <http://dx.doi.org/10.18280/isi.300419>.
- [15] Lu K, Yang Y, Zeng G, Peng C, Geng G, Weng J. BPSO-ahdl-IDS: Binary particle swarm optimization-based automated hybrid deep learning model for intrusion detection of internet of things. *IEEE Trans Autom Sci Eng* 2025;22:15859–77. <http://dx.doi.org/10.1109/TASE.2025.3572510>.
- [16] Kamal H, Mashaly M. Robust intrusion detection system using an improved hybrid deep learning model for binary and multi-class classification in IoT networks. *Technologies* 2025. <http://dx.doi.org/10.3390/technologies13030102>.
- [17] Sajid M, Malik K, Almogren A, Malik T, Khan A, Tanveer J, Rehman A. Enhancing intrusion detection: A hybrid machine and deep learning approach. *J Cloud Comput* 2024;13. <http://dx.doi.org/10.1186/s13677-024-00685-x>.
- [18] Walling S, Lodh S. Network intrusion detection system for IoT security using machine learning and statistical-based hybrid feature selection. *Secur Priv* 2024;7. <http://dx.doi.org/10.1002/spy2.429>.
- [19] Bakır H, Ceviz Ö. Empirical enhancement of intrusion detection systems: A comprehensive approach with genetic algorithm-based hyperparameter tuning and hybrid feature selection. *Arab J Sci Eng* 2024;49:13025–43. <http://dx.doi.org/10.1007/s13369-024-08949-z>.
- [20] Mallidi S, Ramisetty R. Optimizing intrusion detection for IoT: A systematic review of machine learning and deep learning approaches with feature selection and data balancing. *Wiley Interdiscip Rev: Data Min Knowl Discov* 2025;15. <http://dx.doi.org/10.1002/widm.70008>.
- [21] Walling S, Lodh S. A review on feature selection techniques and its significance on IoT IDS. *SN Comput Sci* 2025;6(8):935. <http://dx.doi.org/10.1007/s42979-025-04480-6>.
- [22] Yang L, Shami A. On hyperparameter optimization of machine learning algorithms: Theory and practice. *Neurocomputing* 2020;415:295–316. <http://dx.doi.org/10.1016/j.neucom.2020.07.061>.